

Enoncés TDs et TP / Angular v ≥ 17 , partie 3a

1. TD "PreferencesService" et injection,synchro

Ce TD permettra de mettre en place une première version simple d'un service commun pour partager des données "utilisateur".

Créer le sous **folder** `src/app/common/service` pour bien ranger les services.

Dans un terminal, se placer dans `src/app/common/service` (via `cd` ou bien le menu contextuel *open in integrated terminal* de *Visual-Studio-Code*) et générer un nouveau service via la commande suivante:

```
ng g service preferences --type service
```

Au sein de `PreferencesService`, ajouter (en public) la propriété `couleurFondPreferee` (de type `string`) avec `'lightgrey'` comme valeur par défaut:

`src/app/common/service/preferences.service.ts`

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class PreferencesService {

  public couleurFondPreferee :string = 'lightgrey';

  constructor() { }
}
```

Injecter le service `PreferencesServices` (en public) **dans les constructeurs** de `HeaderComponent` et `FooterComponent`.

NB: Si le Tp est effectué avec une version récente d'angular, on pourra préférer effectuer une injection avec la fonction `inject()` :

```
public preferencesService = inject(PreferencesService) ;
```

Faire en suite en sorte que l'on puisse changer la valeur de `preferencesService.couleurFondPreferee` via une liste déroulante au sein de `FooterComponent`:

`src/app/footer/footer.component.ts` :

```
import { Component, inject } from '@angular/core';
import { PreferencesService } from '../common/service/preferences.service';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-footer',
  imports: [FormsModule],
  templateUrl: './footer.component.html',
  styleUrls: ['./footer.component.scss']
})
export class FooterComponent {

  listeCouleurs : string[] = [ "lightyellow", "white",
    "lightgrey" , "lightgreen" , "lightpink" , "lightblue" ] ;
```

```
//injection moderne via inject()
public preferencesService = inject(PreferencesService) ;

/*
//ancienne injection par constructeur
constructor(public preferencesService : PreferencesService) { }
*/
}
```

src/app/footer/footer.component.html :

```
<p class="piedPage">footer - couleurFondPreferee:
  <select [(ngModel)]="preferencesService.couleurFondPreferee" >
    @for(c of listeCouleurs; track c){
      <option [ngValue]="c">{{c}}</option>
    }
  </select>
</p>
```

Faire en suite en sorte que la nouvelle valeur choisie soit utilisée en tant que couleur de fond de HeaderComponent:

src/app/header/header.component.ts :

```
...
export class HeaderComponent implements OnInit {
  public preferencesService = inject(PreferencesService) ;
  ...
}
```

src/app/header/header.component.html :

```
<p class="entete"
  [style.background-color]="preferencesService.couleurFondPreferee">header
  <a routerLink="/ngr-welcome"> welcome</a> &nbsp;
  <a routerLink="/ngr-basic"> basic</a> &nbsp;
  <a routerLink="/ngr-login"> login</a> &nbsp;
</p>
```

Résultat attendu:

header welcome basic login

welcome to my-app

welcome works!

footer - couleurFondPreferee: lightgreen ▼

Changer de sélection de couleurFondPreferee dans la liste déroulante du pied de page.
La couleur de fond de l'entête devrait normalement être bien synchronisée.

En étant positionné sur une couleur de fond autre que celle par défaut ('lightgrey') , effectuer un "refresh" de la page au sein du navigateur. La valeur de couleurFondPreferee est alors réinitialisée et reprend sa valeur par défaut.

=====

Si l'on souhaite de pas perdre la valeur choisie de couleurFondPreferee au moment d'un "refresh" de la page au sein du navigateur, on peut améliorer le code du service de la façon suivante:

src/app/common/service/**preferences.service.ts**

```
import { Injectable } from '@angular/core';
//import { MyStorageUtilService } from './my-storage-util.service';

@Injectable({
  providedIn: 'root'
})
export class PreferencesService {

  //readonly myStorageUtilService = inject(MyStorageUtilService);

  private _couleurFondPreferee :string ;

  public get couleurFondPreferee(){
    return this._couleurFondPreferee;
  }

  public set couleurFondPreferee(c:string){
    this._couleurFondPreferee=c;
    localStorage.setItem('preferences.couleurFond', c);
    // this.myStorageUtilService.setItemInLocalStorage('preferences.couleurFond',c);
  }

  constructor() {
    //let c :string | null = this.myStorageUtilService.getItemInLocalStorage('preferences.couleurFond');
    let c = localStorage.getItem('preferences.couleurFond');
    this._couleurFondPreferee = c?c:'lightgrey';
  }
}
```

Attention (éventuel ajustement avec angular 17plus et en mode SSR) :

Si une application Angular récente est activée en mode SSR , il faut s'assurer que le code fonctionne bien coté navigateur (en appelant `isPlatformBrowser()`) de manière à pouvoir utiliser l'objet localStorage qui n'est disponible que du coté navigateur (et pas lors d'un pré-rendu coté serveur) :

Ce besoin étant récurrent, autant le coder dans un sous service réutilisable :

```
import { Injectable } from '@angular/core';
import { inject, PLATFORM_ID } from "@angular/core";
import { isPlatformBrowser, isPlatformServer } from "@angular/common";

@Injectable({
  providedIn: 'root'
})
export class MyStorageUtilService {
  private readonly platform = inject(PLATFORM_ID);

  public setItemInLocalStorage(key:string, value: string | null | undefined){
```

```

if (isPlatformBrowser(this.platform)) {
  localStorage.setItem(key,value?"");
}
}

public setItemInSessionStorage(key:string, value: string | null | undefined){
  if (isPlatformBrowser(this.platform)) {
    sessionStorage.setItem(key,value?"");
  }
}

public getItemInLocalStorage(key:string):string|null {
  return isPlatformBrowser(this.platform)?localStorage.getItem(key):null
}

public getItemInSessionStorage(key:string):string|null {
  return isPlatformBrowser(this.platform)?sessionStorage.getItem(key):null
}
constructor() { }
}

```

2. TD/TP simulation d'appels ajax avec Rxjs

Ce TD très important va nous permettre de mettre en place la structure classique d'un appel http/ajax asynchrone (dans un premier temps simulé) depuis une application angular.

2.1. Préparation de la classe de données "Devise"

Réouvrir le projet angular **my-app** dans l'éditeur "Visual-Studio-Code".

Créer (via new File ...) une **nouvelle classe** de données **Devise** dans **src/app/common/data** et comportant les propriétés **.code (string)** , **.name (string)** , **.change (number)**

src/app/common/data/devise.ts

```

export class Devise{
  constructor(public code:string="?",
               public name:string="?",
               public change:number=1){}
}

```

2.2. Préparation du Service "DeviseService"

Générer un nouveau service **DeviseService** dans **src/app/common/service** (et menu contextuel "open in integrated Terminal ...") via la commande

```
ng g service devise --type service
```

Placer le code suivant dans **src/app/common/service/devise.service.ts** :

```

import { Injectable } from '@angular/core';
import { Devise } from '../data/devise';
import { Observable, of } from 'rxjs';

```

```
import { delay, map } from 'rxjs/operators';

@Injectable({
  providedIn: 'root'
})
export class DeviseService {

  //jeux de données (en dur) pour pré-version (simulation asynchrone)
  private devises : Devise[] = [
    new Devise('EUR','euro',1.0),
    new Devise('USD','dollar',1.1),
    new Devise('GBP','livre',0.9)
  ];

  public getAllDevises$() : Observable<Devise[]>{
    return of(this.devises) //version préliminaire (cependant asynchrone)
      .pipe(
        delay(111) //simuler une attente de 111ms
      );
  }

  public convertir$(montant: number,
    codeDeviseSrc : string,
    codeDeviseTarget : string
  ) : Observable<number> {
    let coeff = (codeDeviseSrc===codeDeviseTarget)?1:Math.random();
    //coefficient aleatoire ici (simple simulation)
    let montantConverti = montant * coeff;
    if(montant < 0)
      return throwError( ()=>new Error("montant négatif invalide") );
    return of(montantConverti) //version temporaire (cependant asynchrone)
      .pipe(
        delay(222) //simuler une attente de 222ms
      );
  }
}
```

2.3. Codage du composant "ConversionComponent"

Générer dans **src/app** (et menu contextuel "open in integrated Terminal ..."), un nouveau composant **ConversionComponent** via la commande

```
ng g component conversion --type component
```

Ajouter la ligne suivante dans le tableau des **routes** de **src/app/app-routes.ts** ou bien **app-routing.module.ts**:

```
{ path: 'ngr-conversion', component: ConversionComponent }
```

Ajouter le lien hypertexte suivant dans **src/app/header/header.component.html**:

```
<a routerLink='/ngr-conversion'> conversion</a> &nbsp;
```

Injecter le service **DeviseService** dans **ConversionComponent**.

Placer le code suivant dans **ConversionComponent** de façon à ce que l'on puisse effectuer des conversions de monnaies:

src/app/conversion/conversion.component.ts

```
import { Component, OnInit } from '@angular/core';
import { DeviseService } from '../common/service/devise.service'
import { Devise } from '../common/data/devise'

@Component({
  selector: 'app-conversion',
  imports: [FormsModule],
  templateUrl: './conversion.component.html',
  styleUrls: ['./conversion.component.css']
})
export class ConversionComponent {
  montant: number = 0;
  codeDeviseSource: string = "?";
  codeDeviseCible: string = "?";
  montantConverti: number = 0;

  listeDevises : Devise[] = []; //à choisir dans liste déroulante.

  constructor(private _deviseService : DeviseService) { }

  onConvertir(){
    console.log("debut de onConvertir")
    this._deviseService.convertir$(this.montant,
                                   this.codeDeviseSource,
                                   this.codeDeviseCible)
      .subscribe({
        next : (res :number) => { this.montantConverti = res;
                                console.log("resultat obtenu en différé")} ,
        error : (err) => { console.log("error:"+err)}
      });
    console.log("suite immédiate (sans attente) de onConvertir");
    //Attention : sur cette ligne , le résultat n'est à ce stade pas encore connu
    //car appel asynchrone non bloquant et réponse ultérieure via callback
  }

  initListeDevises(tabDevises : Devise[]){
    this.listeDevises = tabDevises;
    if(tabDevises && tabDevises.length > 0){
      this.codeDeviseSource = tabDevises[0].code; //valeur par défaut
      this.codeDeviseCible = tabDevises[0].code; //valeur par défaut
    }
  }

  ngOnInit(){
    this._deviseService.getAllDevises$()
      .subscribe({
        next: (tabDev : Devise[])=>{ this.initListeDevises(tabDev); },
        error: (err) => { console.log("error:"+err)}
      });
  }
}
```

Adaptation nécessaire en mode "ZoneLess" (par défaut à partir de angular 21) :

En mode moderne "**ZoneLess**" (sans utilisation de l'ancien ZoneJs) , **suite à un traitement asynchrone** (ex : résultat d'un appel http simulé ou réel) les valeurs réactualisées en mémoire et placées dans de simples variables d'instances ne sont **pas** automatiquement **détectées** comme ayant **changées** et devant être rafraîchies du côté affichage html .

Sans signal , il faut **manuellement déclencher** une demande de rafraîchissement via **this.changeDetectorRef.markForCheck();**

en s'appuyant sur un accès au service injecté ChangeDetectorRef :

private changeDetectorRef = inject(ChangeDetectorRef);

Dans le contexte de notre composant , on pourra ajouter la ligne

this.changeDetectorRef.markForCheck();

à la **fin** du code de la méthode **initListeDevises()**

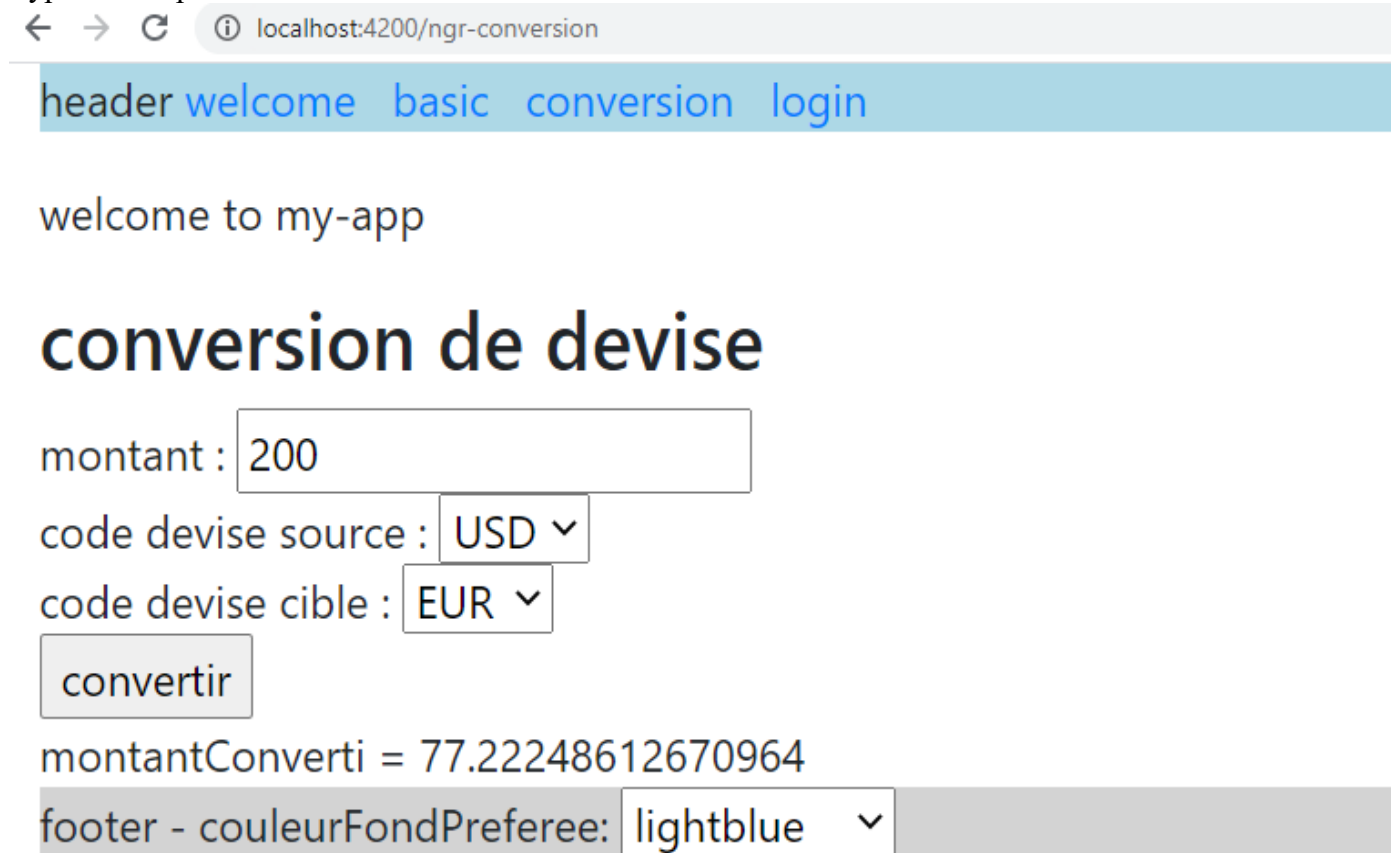
et à la **fin** du code du bloc **subscribe({next : ... } dans onConvertir()**

src/app/conversion/conversion.component.html

```
<h3>conversion de devise</h3>
montant : <input [(ngModel)]="montant" /> <br/>
code devise source : <select [(ngModel)]="codeDeviseSource" >
  @for(d of listeDevises; track d.code){
    <option>{{d.code}}</option>
  }
</select> <br/>
code devise cible : <select [(ngModel)]="codeDeviseCible" >
  @for(d of listeDevises; track d.code){
    <option>{{d.code}}</option>
  }
</select> <br/>
<input type="button" (click)="onConvertir()" value="convertir" /> <br/>
montantConverti = {{montantConverti}}
```

Lancer un test de l'application angular via la commande **ng serve** et un navigateur (<http://localhost:4200>).

Type de comportement attendu:



Ordre des affichages dans la console du navigateur:

debut de onConvertir	conversion.component.ts:22
suite immédiate (sans attente) de onConvertir	conversion.component.ts:31
resultat obtenu en différé	conversion.component.ts:28

Pour expérimenter librement une des variantes suivantes (TD/TP facultatif avec AsyncPipe , avec async/await ou avec signaux) , on pourra effectuer une copie du code actuel de notre composant conversion dans des fichiers ayant de genre de noms :

conversion.component.v1_manual.ts.txt

conversion.component.v1_manual.html.txt

2.4. TD possible (variante avec | async) :

Au sein de **conversion.component.ts** (dans l'état de départ *v1_manual*):

- ajouter **AsyncPipe** dans la partie imports:[...] de @Component()
- ajouter ceci à l'intérieur de la classe ConversionComponent avec une nouvelle version simplifiée du code de onConvertir() :


```
montantConvertiObservable! : Observable<number>
```

```
onConvertir() {  
  this.montantConvertiObservable =  
    this._deviseService.convertir$(this.montant,  
      this.codeDeviseSource,  
      this.codeDeviseCible)  
  //à afficher coté .html via {{ montantConvertiObservable | async }}  
  //ça nécessite imports:[...,AsyncPipe]  
}
```

NB : la syntaxe typescript `variablexyz! : typeVariable` permet de se dispenser d'initialiser une variable d'instance (attribut de la classe) même en mode strict .

Au sein de `conversion.component.html`

remplacer

```
montantConverti = {{montantConverti}}
```

par

```
<!-- montantConverti = {{montantConverti}} -->  
montantConverti = {{montantConvertiObservable | async }}
```

Pas besoin de `changeDetectorRef.markForCheck()` avec `AsyncPipe` même en mode `ZoneLess`

Version améliorée avec gestion d'erreur :

```
.smallError{ color:red; font-size: 10pt; margin-left: 6px; } dans .scss
```

```
<br/><span class="smallError">{{message}}</span> dans .html
```

et dans `conversion.component.ts`

```
message=""  
  
onConvertir() {  
  this.montantConvertiObservable =  
    this._deviseService.convertir$(this.montant,  
      this.codeDeviseSource,  
      this.codeDeviseCible)  
    .pipe(tap((resWithoutErr)=>{this.message=""}),  
      catchError((err)=>this.handleError(err)));  
}  
private handleError(error: any): Observable<never> {  
  this.message="erreur de conversion:" + error; //ou mieux  
  console.log(this.message);  
  //return throwError(()=> new Error('Error ...'));  
  return throwError(()=> error);  
}
```

→ à tester avec une saisie de montant négatif

On pourra éventuellement copier le code dans `conversion.component.v2_asyncPipe.ts_et_html.txt`

NB: il est possible d'automatiser un peu plus la gestion d'erreur avec :

- la partie `.pipe(tap())` , `catchError()` du coté service
- un message d'erreur global (au sens dernière erreur) dans un service partagé
- un affichage global/factorisé du dernier message d'erreur dans un autre composant toujours visible (ex : partie de `HeaderComponent`)

2.5. TP facultatif (appel via async/await et firstValueFrom) :

En partant de `conversion.component.v1_manual.ts_html.txt` ou bien `v2_asyncPipe.ts_et_html.txt`

- Dupliquer le code de la méthode `ngOnInit()`
- renommer `ngOnInitV1` un exemplaire de la versions actuelle (en commentaire ou pas)
- ré-écrire la méthode `ngOnInit()` en s'appuyant sur `async/await` et la fonction `firstValueFrom()` qui permet de transformer un Observable en Promise .

Style de code attendu:

```
async xyz() {  
  try {  
    let resXyz = await firstValueFrom(this._xxxService.zzzzz$(...));  
    ...  
  } catch (err) {  
    console.log(err);  
    ...  
  }  
}
```

NB: en mode `ZoneLess` , on aura encore besoin de `changeDetectorRef.markForCheck()`;
en fin de la sous méthode `initListeDevises` appelée en mode `async/await` en fin de bloc `try` .

On pourra éventuellement effectuer une copie du code dans `v3_async_await.ts.txt`

2.6. TP facultatif (appel via signaux et subscribe)

Avec des fichier `conversion.component.html` et `conversion.component.ts` restitués dans leurs états sauvegardés dans

`conversion.component.v1_manual.ts.txt`

`conversion.component.v1_manual.html.txt`

On pourra coder la nouvelle variante suivante :

- **remplacer** `codeDeviseSource` , `codeDeviseCible`, `montantConverti` et `listeDevises` **par des équivalent basés sur des signaux**
- ne plus avoir besoin d'utiliser `changeDetectorRef.markForCheck()` même en mode `"ZoneLess"`

Exemple :

`sCodeDeviseSource=signal<string>("?");`

`sCodeDeviseCible=signal<string>("?");`

`sMontantConverti=signal<number>(0);`

`sListeDevises =signal<Devise[]>([]);`

et **remanier tout le code** (coté `.ts` et `.html`) avec des adaptations cohérentes en mode `"signal"` .

On pourra éventuellement effectuer une copie du code dans `v4_subscribe_signal.ts_et_html.txt`

3. TD appel Http/REST en mode GET via HttpClient

Ce TD va permettre de passer d'une version simulée à une version réelle du service **DeviseService** (effectuant en interne des **appels ajax/http** vers une **api REST**).

3.1. Vérification préliminaire le l'api REST à invoquer

Code source : <https://github.com/didier-mycontrib/tp-api>

URL de la version en ligne : [https://www.d-defrance.fr/tp/devise-api/...](https://www.d-defrance.fr/tp/devise-api/)

Tester tout particulièrement le comportement des appels suivants:

- <https://www.d-defrance.fr/tp/devise-api/v1/public/devises> devant normalement retourner
[{"name":"Dollar","change":1.109257,"code":"USD"},
{"name":"Yen","change":157.875088,"code":"JPY"},
{"name":"Livre","change":0.844698,"code":"GBP"},
{"name":"Euro","change":1,"code":"EUR"}]
- <https://www.d-defrance.fr/tp/devise-api/v1/public/convert?source=EUR&target=USD&amount=50> devant normalement retourner
{"amount":50,"source":"EUR","target":"USD","result":55.462849999999996}

3.2. Préparation indispensable de l'application angular

Si ancien mode no-standalone (ancienne application angular) :

Ajouter le module technique `HttpClientModule` dans la partie `imports:[...]` de `@NgModule` de `app.module.ts`

Dans application angular récente (en mode standalone) :

Ajouter `provideHttpClient()` dans `app.config.ts` (selon l'exemple du chapitre théorique).

3.3. Première version de l'appel ajax/http en mode get

Effectuer idéalement une copie de `src/app/common/service/devise.service.ts` en `src/app/common/service/devise.service.old.without_http.ts.txt` (ancienne version avec simulation sans appel HTTP).

Première version (à copier/coller) d'un **code fonctionnel** pour `src/app/common/service/devise.service.ts`:

```
import { Injectable } from '@angular/core';
import { Devise } from '../data/devise';
import { Observable, of } from 'rxjs';
import { map } from 'rxjs/operators';
import { HttpClient, HttpParams } from '@angular/common/http';

export interface ConvertRes {
  source :string; //ex: "EUR",
  target :string; //ex: "USD",
  amount :number; //ex: 200.0
  result :number; //ex: 217.3913
}
```

```

};

@Injectables({
  providedIn: 'root'
})
export class DeviseService {

  private _apiBaseUrl = "https://www.d-defrance.fr/tp/devise-api/v1";

  constructor(private _http : HttpClient){}

  public getAllDevises$() : Observable<Devise[]>{
    let url = this._apiBaseUrl + "/public/devises" ;
    console.log( "url = " + url);
    return this._http.get<Devise[]>(url);
  }

  public convertir$(montant: number,
                    codeDeviseSrc : string,
                    codeDeviseTarget : string
                    ) : Observable<number> {

    const url = this._apiBaseUrl + "/public/convert"
      + `?source=${codeDeviseSrc}`
      + `&target=${codeDeviseTarget}&amount=${montant}` ;
    //console.log( "url = " + url);
    return this._http.get<ConvertRes>(url)
      .pipe(
        map( (res:ConvertRes) => res.result)
      );
  }
}

```

Résultats escomptés (après éventuel relance de **ng serve**) , <http://localhost:4200>

← → ↻ localhost:4200/ng-r-conversion

header - app - [welcome](#) basic conversion login

welcome to my-app

conversion de devise

montant :

code devise source :

code devise cible :

montantConverti = 217.39130434782606

footer - couleurFondPreferee:

3.4. Configuration du reverse-proxy de ng serve (TD)

Nous allons maintenant appliquer concrètement le **reverse-proxy** de **ng serve** au niveau de notre application angular.

Créer et coder à la **racine de l'application angular** le nouveau fichier **proxy.conf.json** avec le contenu suivant :

```
{ "/tp/standalone-login-api" : { "secure" : false , "changeOrigin" : true,
                                "target" : "https://www.d-defrance.fr/" } ,
  "/tp/devise-api" : { "secure" : false , "changeOrigin" : true,
                      "target" : "https://www.d-defrance.fr/" }
}
```

Effectuer la modif suivante au sein de `src/app/common/service/devise.service.ts`:

```
//private _apiBaseUrl ="https://www.d-defrance.fr/tp/devise-api/v1";

private _apiBaseUrl ="tp/devise-api/v1";
// with prefix in proxy.conf.json
// (ng serve --proxy-config proxy.conf.json)
// or other config in production mode
```

Repérer le message d'erreur suivant dans la console du navigateur lorsque l'on oublie de relancer ng serve avec l'option `--proxy-config proxy.conf.json` :

```
✖ GET http://localhost:4200/devise-api/public/devise 404
(Not Found)
```

Arrêter si besoin **ng serve** (Ctrl-C) et relancer le via la commande suivante:

ng serve --proxy-config proxy.conf.json

Vérifier le bon fonctionnement de <http://localhost:4200/ngr-conversion>.

Astuce: Pour éviter d'oublier l'option `--proxy-config proxy.conf.json` au démarrage de **ng serve** on peut modifier la partie **serve** du fichier de configuration **angular.json** en y ajoutant l'option **"proxyConfig": "proxy.conf.json"** pour indirectement obtenir de manière automatique les mêmes fonctionnalités.

Configuration partielle de **angular.json** (près la ligne 72):

```
...
"serve": {
  "builder": "@angular-devkit/build-angular:dev-server",
  ...
  "options": {
    "proxyConfig": "proxy.conf.json"
  }
}
...
```

Avec cette configuration, un démarrage simple de **ng serve** (*sans option*) suffit et l'on bénéficie tout de même de la fonctionnalité "reverse proxy".

4. TP appel Http/REST en mode POST

Générer un nouveau service **LoginService** dans **src/app/common/service** en lançant la commande suivante en étant positionné au bon endroit dans un terminal:

ng g service login --type service

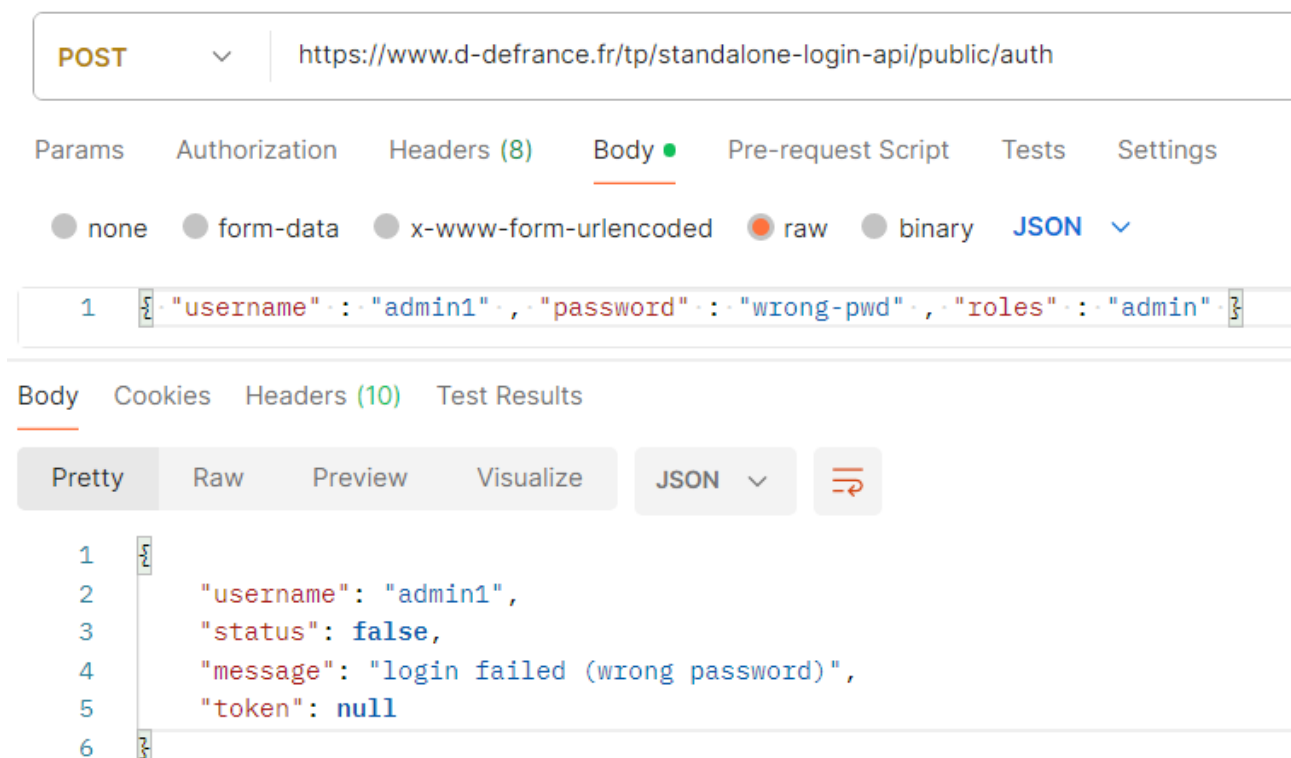
Injecter ce service dans **LoginComponent** (du tp "validation formulaire")

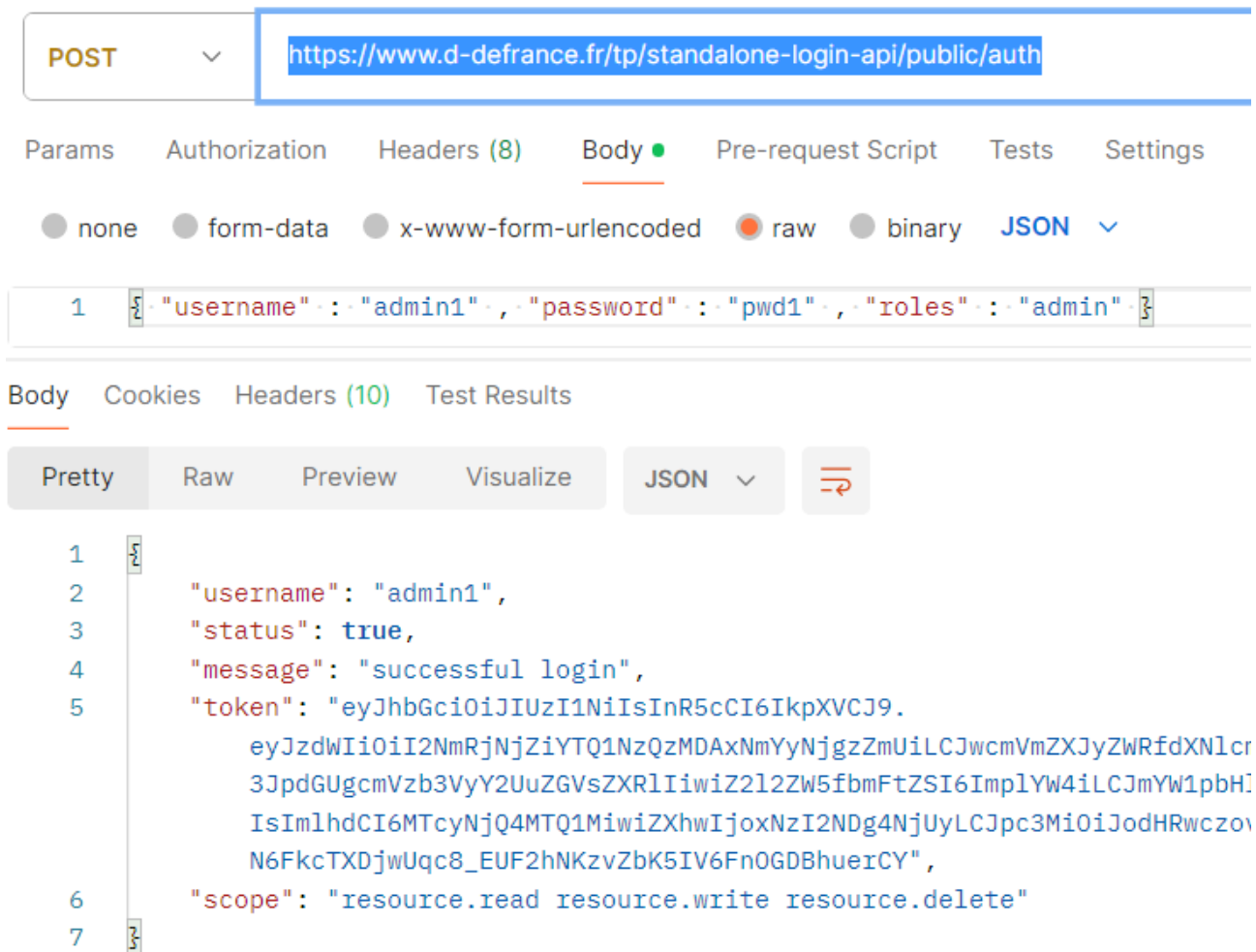
Voici quelques copier/coller des résultats d'un test "PostMan" pour comprendre le fonctionnement du Web-service dont l'URL est <https://www.d-defrance.fr/tp/standalone-login-api/v1/public/auth> qu'il est prévu d'invoquer en mode **POST** avec **"Content-Type" : "application/json"** et avec les données *raw* suivantes:

```
{ "username" : "admin1" , "password" : "wrong-pwd" , "roles" :  
  "admin" }
```

ou bien

```
{ "username" : "admin1" , "password" : "pwd1" , "roles" :  
  "admin" }
```





Coder ensuite (dans `src/app/common/data`) , une nouvelle classe **LoginResponse** correspondant à la structure de la réponse renvoyée par le web service d'authentification.

Coder ensuite la méthode suivante dans **LoginService**:

```
public postLogin$(login: Login): Observable<LoginResponse>
en s'appuyant sur this._http.post<LoginResponse>( ... )
```

Appeler ensuite cette méthode **postLogin\$(...)** au sein de **doLogin()** de **LoginComponent** avec un **.subscribe()** et des **callbacks** sous forme de **lambdas/arrow-functions**.

Bien afficher le message (message positif ou message d'erreur, ...) à destination de l'utilisateur.

Afficher également toute la réponse reçue au format **JSON** dans les **logs** du navigateur.

Arrêter et relancer si besoin `ng serve` et **tester l'application**.

résultats attendus(<http://localhost:4200/ng-r-login>):

```
loginResponse={ "username": "admin1", "status": true, "message": "successful
login", "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWUiOiI2NmRjNjZiYTQ1NzQzMdAxNmYyNjgzZmUiLCJwcmVmZXJyZWRfdXNlcr3JpdGUGcmVzb3VyY2UuZGVsZXRLIiwiaWZ2L2ZW5fbmFtZSI6ImplYW4iLCJmYW1pbHlIsIm1hdCI6MTcyNjQ4MTQ1MiwiZXhwIjoxNzI2NDg4NjUyLCJpc3MiOiJodHRwczovLjN6FkcTXDjwUqc8_EUF2hNKZvZbK5IV6FnOGDBhuerCY", "scope": "resource.read resource.write resource.delete" }
```

login

username:

admin1

password:

pwd1

roles:

admin

submit/login 

successful login

ou bien (si mauvais mot de passe):

login failed (wrong password)

NB: En version angular>=21 (par défaut en mode zoneLess) , il faudra soit penser à appeler `this.changeDetectorRef.markForCheck()` ou bien soit transformer la variable message en signal .

5. TP prise en compte de l'authentification

Si le Tp complémentaire facultatif "Devise_CRUD" (partie 3b) a été effectué complètement (ce qui demande du temps) , on dispose alors déjà d'un composant permettant de modifier une devise.
Si par contre le TP complémentaire facultatif "Devise_CRUD" (partie 3b) n'a pas été effectué , on compensera alors ceci par l'ajout de la version minimaliste suivante:

ng g component devise --type component

Ce nouveau composant "devise" au sens "mise à jour devise" servira à modifier le cours d'une devise :

code	nom	change
EUR	Euro	1
USD	Dollar	1.1
GBP	Livre	0.9
JPY	Yen	123.8

code devise a modifier: (ex: JPY , GBP)

nouveau change: (ex: 0.9 , 1.1)


```
{"headers":{"normalizedNames":{},"lazyUpdate":null},"status":401,"st  
v=true: 401 Unauthorized","error":"Unauthorized"}
```


Pour coder cela rapidement, on pourra ajouter les fonctions suivantes dans `src/app/common/service/devise.service.ts`

```
...
public getDeviseByCode$(code :string ) : Observable<Devise>{
  let url = `${this._apiBaseUrl}/public/devises/${code}` ;
  console.log( "url = " + url);
  return this._http.get<Devise>(url);
}

putDevise$(d :Devise): Observable<Devise>{
  //const url = `${this.publicOrPrivateBaseUrl}/devises/${d.code}?v=true`;
  const url = `${this._apiBaseUrl}/private/devises/${d.code}?v=true`;
  return this._http.put<Devise>(url,d /*input envoyé au serveur*/);
}
```

devise.component.css

```
table , th, td { border : 1px solid rgb(146, 16, 146); }
```

Pour le contenu de *devise.component.ts* et *.html* , on pourra soit rechercher la solution par soi même , soit s'inspirer de l'exemple de code suivant :

devise.component.ts

```
import { Component, inject, signal } from '@angular/core';
import { firstValueFrom, Observable } from 'rxjs';
import { Devise } from '../common/data/devise';
import { DeviseService } from '../common/service/devise.service';
import { FormsModule } from '@angular/forms';
import { AsyncPipe } from '@angular/common';

@Component({
  selector: 'app-devise',
  imports: [FormsModule, AsyncPipe],
  templateUrl: './devise.component.html',
  styleUrls: ['./devise.component.css'],
})
export class DeviseComponent {

  private _deviseService = inject(DeviseService);

  message=signal("");
  codeToUpdate="?";
  changeToUpdate=1;

  devises$! : Observable<Devise[]>;

  async onRefresh() {
    this.devises$ = this._deviseService.getAllDevises$();
  }

  ngOnInit(){
    this.onRefresh();
  }
}
```

```

async onUpdate() {
  try {
    let d:Devise;
    let deviseTemp : Devise|undefined;
    deviseTemp = await firstValueFrom(
      this._deviseService.getDeviseByCode$(this.codeToUpdate));
    deviseTemp.change=this.changeToUpdate;
    await firstValueFrom(this._deviseService.putDevise$(deviseTemp));
    this.message.set("mise à jour ok");
    this.onRefresh();
  } catch (err) { console.log(err); this.message.set(<string> JSON.stringify(err));
  }
}

```

devise.component.html

```

<table>
  <thead><tr><th>code</th><th>nom</th><th>change</th></tr></thead>
  <tbody>
    @for(d of devises$ | async ; track d.code){
      <tr><td>{{d.code}}</td><td>{{d.name}}</td><td>{{d.change}}</td></tr>
    }
  </tbody>
</table>
<hr/>
code devise a modifier: <input [(ngModel)]="codeToUpdate" /> (ex: JPY , GBP) <br/>
nouveau change: <input [(ngModel)]="changeToUpdate" /> (ex: 0.9 , 1.1 ) <br/>
<button (click)="onUpdate()">update devise</button>
<hr/>{{message()}}

```

Il faudra penser à ajouter la route allant vers le composant "devise" dans *app.routes.ts* et une navigation dans *src/app/header/header.component.html* .

Normalement, une première tentative de "update" sans authentification doit se solder par un échec (401/ Unauthorized) .

5.1. TP Authentification en mode "standalone" (sans OAuth2)

Améliorer le code de **onLogin()** (au sein de **login.component.ts**) en faisant en sorte que :

- à chaque nouvelle tentative de login, le contenu de "access_token" soit réinitialisé à "" au sein de sessionStorage du navigateur
- que le contenu de "access_token" au sein de **sessionStorage** soit remplacé par **loginResponse.token** à l'issue de la tentative de login

Créer un nouveau répertoire d'organisation *src/app/common/interceptor*

Se placer dedans (au sein d'un terminal texte) et lancer la commande suivante :

```

ng g interceptor my-auth

```

Ajuster le contenu de la fonction d' **interception** avec **paramètres**(req,next) avec un code de ce genre :

```
//NB: "access_token" plutôt que "token" or "authToken" for angular-oauth2-oidc extension compatibility
const token = sessionStorage.getItem('access_token');
if(token && token !== "" && token !== "null"){
  const authReq = req.clone({
    headers: req.headers.set('Authorization', 'Bearer ' + token)
  });
  console.log("MyAuthInterceptor , adding Bearer token="+token)
  return next(authReq); //ou bien next.handle(authReq) dans anciennes versions d'angular
}else
  return next(req); //ou bien next.handle(req) dans anciennes versions d'angular
```

Enregistrer cet intercepteur au sein de **src/app/app-config.ts** ou bien **app.module.ts** selon les indications du support de cours théorique (fin chapitre HTTP/REST) , selon la version de angular utilisée et selon le contexte (standalone ou pas).

Exemple (pour version récente d'angular) au sein de **src/app/app-config.ts** :

```
...
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { myAuthInterceptor } from './common/interceptor/my-auth.interceptor';
export const appConfig: ApplicationConfig = {
  providers: [... , provideHttpClient( withInterceptors( [myAuthInterceptor] ) )
  ] };

```

Relancer éventuellement **ng serve** pour une meilleur prise en compte de ces changements.

Nouveau comportement attendu :

- Après un login en échec (ex : mauvais mot de passe) , on ne doit pas pouvoir modifier une devise (401/Unauthorized) . Cette erreur être peut être visible dans la console web du navigateur...
- Après un login réussi (avec compte admin1/pwd1) , on doit pouvoir modifier une devise sans erreur
- Après un login réussi (avec compte user1/pwd1) avec un compte n'ayant pas assez de droits/privilège , on ne doit pas pouvoir modifier une devise (403/Forbidden) . Cette erreur être peut être visible dans la console web du navigateur...

NB : on pourra facultativement ajouter un bouton **logOut** qui servirait à réinitialiser à vide 'access_token' dans sessionStorage .

5.2. TD facultatif Authentification en mode OAuth2

En appliquant tout le mode opératoire du paragraphe "accès à un serveur d'autorisation depuis angular" du chapitre "Authentification au sein d' Angular" , on doit pouvoir mettre en place une seconde variante de l'authentification basée sur une délégation d'authentification vers un serveur d'autorisation (ex : "https://www.d-defrance.fr/keycloak/realms/sandboxrealm") .

```
npm install angular-oauth2-oidc --save
```

NB : Pour éviter toute confusion entre les versions "standalone" et "oauth2" du login , on pourra éventuellement utiliser les noms suivants :

- src/silent-refresh.html

- `src/app/common/data/UserInSession` (avec username, authenticated, roles et grantedScopes)
- `src/app/common/service/OAuth2SessionService` (avec utilisation de angular-oauth2-oidc et méthode `initOAuthServiceForCodeFlow()`)
- `src/app/oauth2-log-in-out (OAuth2LogInOutComponent)`
à créer via `ng g component oauth2LogInOut`

NB: La plupart des fichiers nécessaires se trouvent au sein du répertoire "pour_debut_tp".
Quelques copier/coller peuvent éventuellement servir à gagner un peu de temps ...

Ajouter une copie de **silent-refresh.html** in **/src**
et ajouter "src/silent-refresh.html" dans le premier bloc assets de **angular.json** (près de la ligne 30 ou 40)

```
"assets": [
  {
    "glob": "**/*",
    "input": "public"
  },
  "src/silent-refresh.html"
],
```

En version "avec module" , ajouter **OAuthModule.forRoot()** dans imports :[] de **app.module.ts** avec import { OAuthModule } from 'angular-oauth2-oidc';

En version "standalone" , ajouter **provideOAuthClient()** dans providers:[] de **app.config.ts** avec `import { provideOAuthClient } from 'angular-oauth2-oidc';`

Navigations envisageables:

```
...
{ path: "ngr-login" , component : LoginComponent},
{ path: "ngr-oauth2-login-out" , component : OAuth2LogInOutComponent},
{ path: "ngr-devise" , component : DeviseComponent},
{ path: "ngr-conversion" , component : ConversionComponent},
...
```

et

[illegible]

L'intercepteur est le même que celui déjà configuré en mode standalone .

Un arrêt/redémarrage de ng serve sera nécessaire .

En cas de soucis/bug (dans la communication avec le serveur OAuth2) une manip navigateur "Effacer les cookies et les données du site" peut souvent suffire à remettre les choses en place.

5.3. TP différé – authentification peaufinée avec gardien et message

On essaiera (en temps utile , au sein des futurs Tps facultatifs) de peaufiner les points suivants :

- nouveau service "sessionService" permettant de mémoriser des informations sur l'utilisateur connecté (soit via le login standalone , soit via le login oauth2)

-
- Affichage des informations sur la personne connectée (username , ...) au sein de footerComponent
 - gardien bloquant la navigation vers "devise" (ou a défaut "conversion") lorsque l'utilisateur n'est pas connecté . NB : ce tp sera différé dans l'attente de la théorie sur les gardiens (chapitre "routing évolué").

6. TP facultatif sur BehaviorSubject ou signal

Technologie "BehaviorSubject" n'est conseillée que sur des anciens projets "angular" .
Technologie "Signal" (plus moderne, plus performante) conseillée sur projet récent.

- (nouveau) service "sessionService" permettant de mémoriser des informations sur l'utilisateur connecté (soit via le login standalone , soit via le login oauth2) avec par exemple `public sUserInSession = signal( new UserInSession());` où UserInSession est une classe rangée dans `src/app/common/data$user_in_session.ts`
- Ce service comportera des informations en mode "BehaviorSubject" ou bien "signal"
- Affichage d'un icône "connecté" ou pas "connecté" au sein du composant "header"
- Affichage des informations sur la personne connectée (username , ...) au sein de footerComponent
- Ces affichages seront automatiquement synchronisés via des ".subscribe()" " associés au behaviorSubject ou bien via les automatismes des signaux